

Federated Commerce on nopCommerce

Scenario A · Architectural Evolution After Acquisitions

Software Architecture · Group Assignment 02 · Part 2

Ricardo Dias **108598** · Sérgio Correia **108647**
June 2026

The Architectural Tension

*Northstar Living Group: a retail holding that grew by acquisition.
Two specialist BUs need a shared platform without losing local autonomy.*

PULL • CENTRALISATION

- ✓ Single sign-on across the group
- ✓ One platform
- ✓ Group-level customer view and analytics

PULL • ISOLATION

- ✓ BU-specific pricing and catalog rules
- ✓ Local ERP, stock, order workflows
- ✓ Independent deployability, no failure crossover

No neutral position. Every share-vs-isolate choice has consequences for coupling, failure propagation, and BU autonomy.

From Design to a Running System

1 **Per-BU isolation, deployed** [ADR-001](#) · [QA1](#) · [QA2](#)
Two nopCommerce processes, two private PostgreSQL databases. No shared state across the boundary.

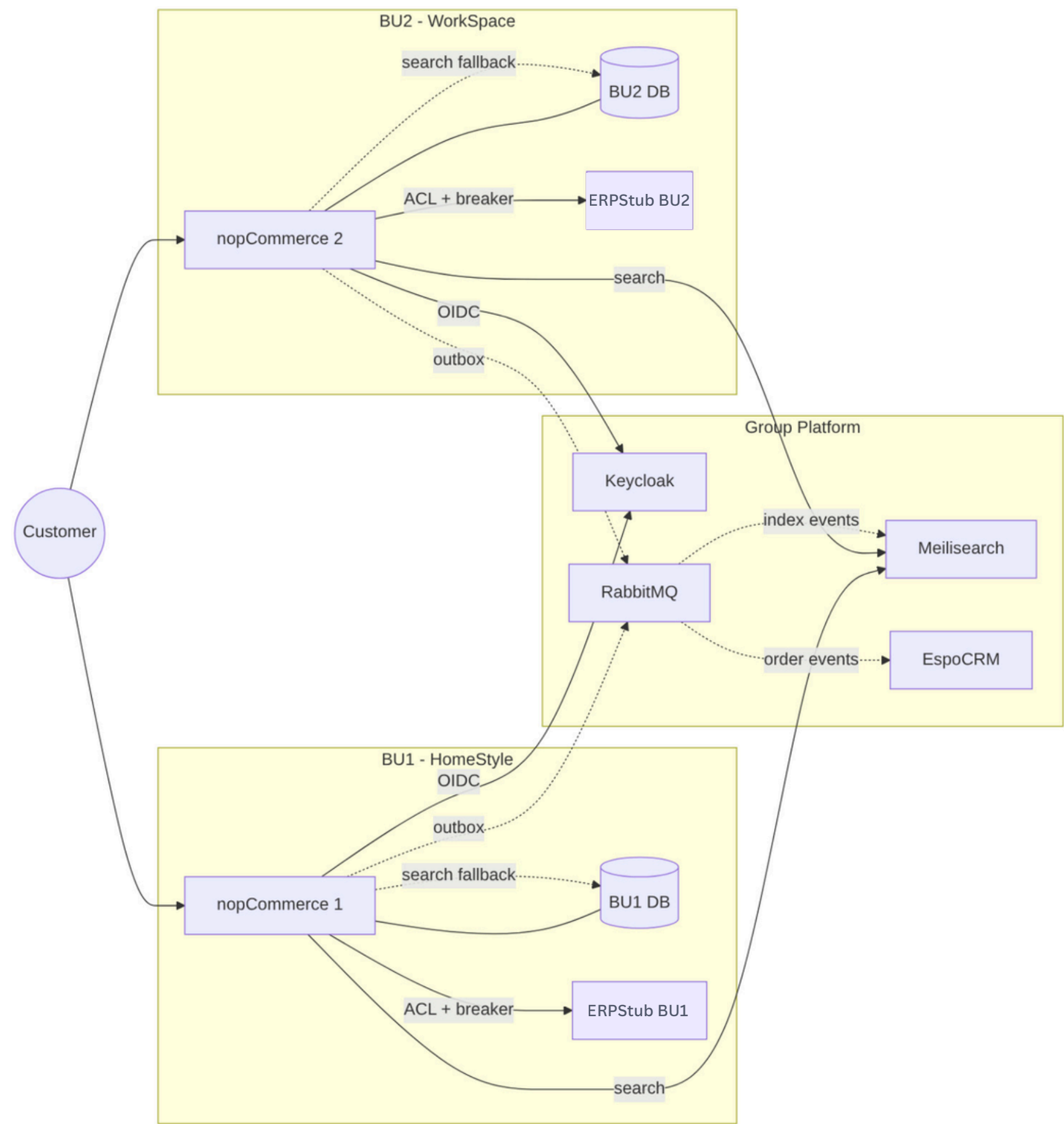
2 **Federated identity, live** [ADR-002](#) · [QA3](#)
Keycloak group realm. One OIDC login flows to both storefronts with no re-prompt.

3 **Cross-BU async coordination** [ADR-003](#) · [QA5](#)
Transactional outbox to RabbitMQ. A worker upserts each order into EspoCRM, idempotently.

4 **BU-local ERP behind a breaker** [ADR-004](#) · [QA1](#)
Polly circuit breaker per BU. Open-circuit serves cached stock with a labelled banner. The pressure point.

5 **Federated search with fallback** [ADR-005](#) · [QA4](#)
Meilisearch primary, BU-local SQL fallback on timeout. No customer-visible 5xx.

What We Built · The Runnable Environment



Two BU storefronts, four group services, two ERP stubs, one portal.
Eight independently deployable subsystems · no shared DB · up with one compose command.

First-Party Code · Plugins & Workers

ADDITIVE. EVERYTHING LIVES IN A PLUGIN OR A WORKER.

Four nopCommerce plugins

- ExternalAuth.Keycloak - per-BU OIDC login
- OutboxRelay - order hook + RabbitMQ relay
- ErpIntegration - circuit breaker + stock banner
- Search.Meilisearch - provider + SQL fallback

Workers & group services

- EspoCrmConsumer - standalone .NET worker
- ErpStub - /admin/break + /admin/recover
- Keycloak · RabbitMQ · Meilisearch · EspoCRM
- Prometheus + Grafana observability spine
- nginx portal - cross-BU landing page

Design to Code · Traceability

Every Part 1 decision maps to a concrete artifact in the repo and a script that proves it.

DECISION	WHAT WE BUILT	VERIFIED BY
Per-BU process + DB isolation	2 nopCommerce processes, 2 Postgres	test-isolation.sh · 29/29 HTTP 200
Keycloak group SSO (OIDC)	ExternalAuth.Keycloak plugin	test-sso.sh · one login, both BUs
Transactional outbox to CRM	OutboxRelay plugin + EspoCrmConsumer	demo-outbox.sh · 100 queued, 0 lost
Circuit breaker around BU-local ERP	ErpIntegration plugin + ErpStub	demo-breaker.sh · state 0-2-1-0
Federated Meilisearch + SQL fallback	Search.Meilisearch plugin	demo-search-fallback.sh · 0 5xx

Five ADRs, five proofs. Each row is reproducible with one script.

Live Demo

Limitations & Deferred for Production

ADR-003

In-memory idempotency

The consumer tracks processed orders in memory; a restart clears it, risking a repeat.

Fix: persist processed keys in EspoCRM or a table.

ADR-005

Timeout, not a breaker

Every search waits 1.5s before using the backup, instead of skipping straight to it.

Fix: wrap in a Polly breaker like the ERP.

VENDOR

Upgrade-fragile patches

Three minimally invasive edits to ProductService need re-applying on any nopCommerce upgrade.

DEPLOY

Single host, single instance

All containers run on one Docker host. Scaling out (HA Keycloak, RabbitMQ cluster) needs no code change.

OBSERVABILITY

Metrics, not tracing

Prometheus aggregates, but there is no per-order distributed trace.

Fix: W3C TraceContext across the async hops.

Questions?